

Especificaciones de Casos de Uso

Un caso de uso (CU) describe una secuencia de interacciones entre un sistema y un actor, que resulta en el actor obteniendo alguna salida de valor del sistema.

La especificación de casos de uso consiste en brindar detalles de lo que el CU debe realizar, para poder brindar esa funcionalidad que da valor al actor que lo activa.

En este texto se presentará y describirá la plantilla (template) propuesta por la materia para utilizar en la resolución de las guías de ejercicios.

Cada organización o equipo puede elegir o acordar el uso de un template determinado. Lo que se propone aquí es una guía que abarca gran cantidad de información útil, pero no debe considerarse un listado exhaustivo de las secciones que el template debe tener. Es posible agregar nuevas partes que no son mencionadas aquí, así como también quitar alguna que no se considere necesaria.

A continuación, se muestra la plantilla propuesta y luego se describe el objetivo de cada sección. Además, se brindan algunos ejemplos para describir las distintas opciones que pueden considerarse a la hora de especificar casos de uso.

Template propuesto

ID		
Nombre		
Breve descripción		
Prioridad		
Complejidad		
Actor principal		
Actores secundarios		
Precondiciones		
Postcondiciones	Éxito:	
	Fracaso:	
Flujo Principal	Flujo Alternativo	
Flujos de excepción		
Observaciones		

ID

Cada caso de uso debe tener una identificación (ID) que permita distinguirlo de los demás. Esta identificación es única en toda la especificación de casos de uso, es decir, no puede haber dos casos de uso con el mismo ID dentro de una misma especificación.

Con respecto al formato, en general, lo decide la organización o el equipo. Puede ser un identificador numérico, alfanumérico, o cualquier otro esquema que el equipo decida utilizar. Lo importante es que, una vez que se decida un esquema de identificación, se utilice de manera consistente.

Ejemplos de identificadores:

- CU-001
- CU_001
- 001

Se puede pensar que este campo no es necesario, y que podría utilizarse solamente el nombre del caso de uso para identificarlo. Pero el problema con este enfoque es que la especificación puede evolucionar, lo cual puede hacer que el nombre de un CU cambie con el tiempo y nos perjudique a la hora de su identificación. En cambio, el ID es estático, una vez asignado, no es posible cambiarlo y facilita la gestión de la trazabilidad de los CU y los requerimientos asociados.

Nombre

El nombre de un caso de uso se escribe, generalmente, utilizando un verbo o una frase verbal seguida de un sustantivo. El nombre debe ser claro y conciso, y debe transmitir la acción de valor que el mismo realizará mediante la funcionalidad que representa. Debe elegirse según la visión del usuario o cliente y no del analista o desarrollador. Está orientado a que el usuario sobreentienda el objetivo del CU a través de su nombre.

Ejemplos de nombres adecuados:

- Agregar Cliente
- Dar de alta Proveedor
- Exportar listado de Clientes morosos
- Generar reporte de Ventas mensuales

Vale la pena resaltar que, dentro de una misma especificación, debemos intentar ser consistentes y utilizar la misma frase verbal si estamos describiendo la misma acción (recordemos que la redundancia es bienvenida!). Por ejemplo, en el listado anterior, vemos que se utiliza el verbo *Agregar* primero y luego *Dar de alta*. Esto ocasiona que el lector se pregunte, “¿cuál es la diferencia entre estos dos casos de uso?”. Para evitar este “trabajo extra” de tener que contestar esa pregunta, tenemos que usar siempre el mismo verbo, ya sea *Agregar cliente* y *Agregar proveedor*, o bien *Dar de alta cliente*, y *Dar de alta proveedor*. De esta forma, se transmite claramente que la acción es la misma en los dos casos.

Breve descripción

En este campo encontramos una breve descripción textual del propósito del caso de uso. La idea es que, leyendo la descripción, se pueda entender el objetivo del caso de uso. Obviamente, este campo no brindará todos los detalles del mismo, pero sí debe especificar el objetivo.

La especificación de todos los casos de uso de un proyecto puede generar un documento muy extenso, y como tal, puede ser difícil de manejar. En algunos casos, sobre todo cuando se producen acuerdos contractuales entre el cliente y el proveedor del software, se desea obtener alguna forma de resumen de aquellas funcionalidades que estarán implementadas en el sistema. En estas ocasiones, se genera un documento que, de cada caso de uso, sólo incluye los campos *ID*, *Nombre* y *Descripción*. Esto, junto con el

diagrama de casos de uso, forma un documento más sencillo de manejar que muestra cada funcionalidad que el sistema va a realizar.

Algunos ejemplos:

Caso de uso	Descripción
Cancelar órdenes erróneas	El caso de uso realiza la cancelación de aquellas órdenes que hayan resultado erróneas e informa al sistema actual de procesamiento de pedidos.
Agregar Producto	El caso de uso permite la creación de un producto en el sistema, para que el mismo esté disponible en la operación de venta.
Consultar Proveedores	Permite al usuario hacer una búsqueda en el sistema de uno o varios Proveedores, según determinados criterios de búsqueda.

Como se ve en los ejemplos, no es necesario entrar en demasiado detalle, simplemente es especificar el objetivo del caso de uso, un poco más allá de lo que se muestra en el nombre del mismo.

Prioridad

Ningún recurso es infinito en un proyecto de software. Esto quiere decir que contaremos con un número acotado de miembros del equipo (programadores, testers, diseñadores gráficos, etc.), así como también tendremos fechas de entrega que cumplir. En la gran mayoría de los casos, la cantidad de funcionalidades que hay que implementar en el sistema sobrepasa la capacidad de esos recursos. También suele darse que el cliente necesite obtener una implementación rápidamente (aunque mínima) para poder llegar antes al mercado. En cualquier caso, es claro que no todas las funcionalidades son igualmente importantes o necesarias.

Este campo permite representar la importancia de cada caso de uso para poder decidir qué funcionalidad debe implementarse primero.

La escala utilizada aquí debe ser acordada entre el equipo y el cliente, y debe ser utilizada de manera consistente en todo el proyecto. En este campo no se deben brindar explicaciones textuales, sino simplemente se asigna un valor asociado a una escala de prioridades.

Normalmente, en la materia utilizamos la siguiente escala:

- **Alta:** lo más importante.
- **Media:** medianamente importante.
- **Baja:** menor importancia.

La idea es comenzar el trabajo por aquellos Casos de Uso de prioridad alta primero, luego los de prioridad media, y así sucesivamente.

Complejidad

Muchas veces, durante el desarrollo de un proyecto de software, se trabaja con iteraciones. Períodos de cierta cantidad de tiempo, en los que se trabaja en la implementación de un subconjunto de los casos de uso. Muchas veces sucede que se necesita conocer una estimación del tiempo que llevará cada funcionalidad para ser implementada. Esta estimación, no es fácil de obtener, dado que en etapas tempranas no se conoce demasiado sobre esto.

La complejidad es un campo muy parecido a la Prioridad, dado que permite especificar una escala, pero esta vez, la escala representa la dificultad que implica implementar esta funcionalidad.

A la hora de estimar una funcionalidad, se puede usar este campo. Por ejemplo, se puede asumir que una tarea de complejidad *Alta* llevará más tiempo que una de complejidad *Baja*.

Para la materia utilizaremos la siguiente escala:

- Alta
- Media
- Baja

Nuevamente, cabe destacar que la escala a utilizar debe ser acordada por el equipo, y también utilizada de manera coherente para todos los casos de uso.

Actor Principal

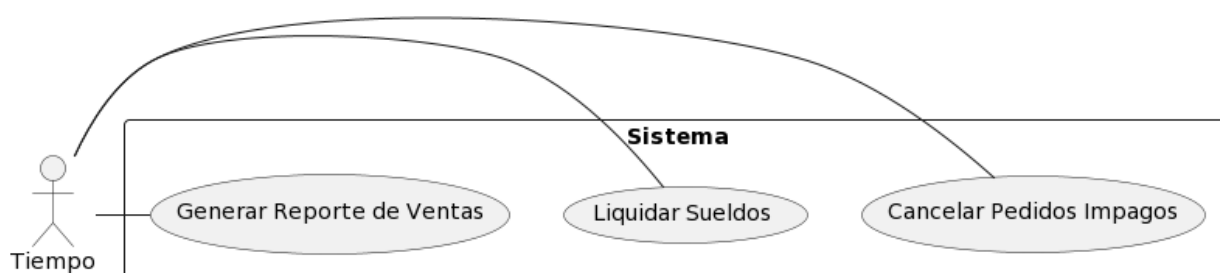
Como se sabe, los agentes externos que interactúan con los casos de uso, ya sea activándolos o participando en la realización de alguno de ellos, son denominados Actores. Estos agentes toman el rol de Actor Principal para un caso de uso, si participan en la activación del mismo. Es decir, el Actor Principal es el que invoca o desencadena la ejecución de un caso de uso.

El Tiempo como Actor Principal

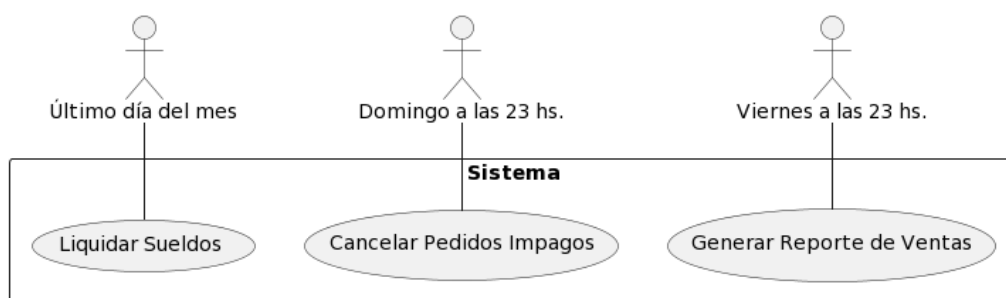
Muchas veces existen casos de uso que son activados de forma automática por el sistema, dependiendo de alguna expresión temporal, como por ejemplo “El día 1 de cada mes”, “Al finalizar cada bimestre”, “Diariamente a las 23 hs”, etc. Claramente, en estos casos, el actor que activa el caso de uso es el tiempo.

Para representar el tiempo como actor principal, tenemos dos opciones:

- Utilizar el nombre *Tiempo*:
 - Todos los casos de uso activados por el tiempo están relacionados con el mismo actor.
 - La especificación de cada caso de uso contiene la expresión de tiempo que causa la activación del caso.
 - La ventaja principal de este enfoque es que, un cambio en la expresión de tiempo no afecta el diagrama de casos de uso.



- Utilizar como nombre de actor, la expresión de tiempo
 - Hay más de un actor representando el tiempo.
 - La ventaja de este enfoque es que, mirando el diagrama de casos de uso, ya puede conocerse en qué momentos estos casos de uso serán generados.



Actores Secundarios

En ocasiones, se encontrarán casos de uso que, como parte de la ejecución, interactúan con otro actor, o con un sistema externo para realizar alguna validación, realizar algún pedido, etc. En estos casos, los sistemas externos representan actores, pero no son actores primarios, dado que no activan al caso de uso. Estos actores son Secundarios, es decir, participan de la ejecución de un caso de uso, pero no activan al mismo.

Un ejemplo frecuente es un caso de uso para realizar el pago con una tarjeta de crédito, en donde nos encontraremos que, en medio de la funcionalidad, el sistema se comunica con un sistema bancario para poder realizar el pago con la tarjeta.

Precondiciones

Las precondiciones definen prerequisites que deben cumplirse para poder ejecutar el caso de uso. El sistema debe ser capaz de verificar estos requisitos, y si todos ellos son verdaderos (es decir, se cumplen), entonces se puede ejecutar el caso de uso.

Las precondiciones hacen referencia al estado del sistema al momento de ejecutar la funcionalidad. Si el estado del sistema no es el esperado, entonces la funcionalidad no se ejecuta.

Algunos ejemplos de precondiciones:

- El usuario debe haber iniciado sesión en el sistema.
- Las inscripciones a las materias de la carrera deben estar abiertas (por ejemplo, para que un alumno pueda inscribirse a una materia).

PostCondiciones

De manera similar, las postcondiciones describen el estado resultante del sistema, luego de ejecutar el caso de uso. Esto significa que, si el caso de uso se ejecuta correctamente, el estado resultante será el descrito en las postcondiciones.

Las postcondiciones pueden representar, además del estado del sistema, algo observable por el usuario, o también, alguna salida física del sistema (por ejemplo, en un cajero automático podríamos decir como postcondición que el dinero es entregado al usuario).

Existen dos tipos de postcondiciones:

- **Postcondiciones de éxito:** describen el estado resultante del sistema, luego de que el caso de uso se ejecute de forma exitosa. El término "exitoso" aquí debe tomarse con cuidado, porque lo que significa es que se haya cumplido el objetivo del caso de uso.
- **Postcondiciones de fracaso:** describen el estado resultante de una ejecución no exitosa. Que la ejecución no sea exitosa, no quiere decir que haya habido una interrupción en la misma, sino que el objetivo del caso de uso no se pudo alcanzar, probablemente dado por el fallo de alguna validación. Hay que tener en cuenta que el fallo de una validación es algo esperable, para lo cual el caso de uso estará preparado, y la ejecución debe seguir hasta finalizar, pero sin cumplir el objetivo.

Por ejemplo, si nuestro caso de uso es el de Agregar Cliente, típicamente tendremos las siguientes postcondiciones:

- **Éxito:** el nuevo cliente queda registrado en el sistema.
- **Fracaso:** no se registra el nuevo cliente.

En el ejemplo podemos ver que, en ambos casos, el caso de uso se ejecuta bien, finaliza dentro de carriles esperados, no hay ninguna interrupción, pero el objetivo puede no cumplirse.

Flujo Principal

Dentro de la especificación del caso de uso encontramos el flujo principal, que es el escenario del mismo que representa el caso de éxito, también conocido como *happy path*. Este escenario describe, en una lista numerada de pasos, la interacción entre el actor y el sistema, que llevan a una realización exitosa del caso de uso, en la cual se cumple el objetivo del mismo, es decir, se cumple con la postcondición de éxito.

Típicamente, el primer paso del caso de uso que se encuentra en la plantilla corresponde a lo que se conoce como **activación del caso de uso**. Este paso demuestra cómo es invocado el mismo. Algunos ejemplos:

- El caso de uso comienza cuando el usuario presiona el botón "Agregar Cliente".
- El caso de uso comienza cuando el usuario presiona el botón "Modificar producto" sobre un producto seleccionado.
- El caso de uso comienza todos los viernes a las 18 hs.

Mayormente, lo que vamos a encontrar es una interacción entre el actor y el sistema, en donde el actor realiza una acción (paso 1), el sistema realiza otra (paso 2), y así sucesivamente.

Dentro del flujo principal, podemos encontrar algunos constructores que son de amplio uso, tales como: condicionales, ciclos, inclusiones y puntos de extensión, entre otros. Veremos estos constructores a continuación.

Para mantener una consistencia en la especificación, tenemos que tener en cuenta lo siguiente:

- la ejecución completa del flujo principal debe coincidir con la postcondición de éxito (con todas, si es que hay más de una).
- podemos asumir que todas las precondiciones se cumplen.

Condicionales

Los condicionales son decisiones o expresiones que deben evaluarse, las cuales pueden ser verdaderas o falsas. Estas condiciones, dependiendo del resultado de la evaluación, definen un camino de ejecución u otro. Es decir, el comportamiento cambia, en base al resultado de la condición.

Es posible plantear un condicional como parte del flujo principal, siempre y cuando la ejecución del flujo continúe siendo parte del escenario principal.

A continuación, veremos algunos ejemplos de condicionales en el flujo principal. Pero lo que debemos recordar es que tenemos que cubrir todas las alternativas que haya, dado que si no lo hacemos, hay caminos que el caso de uso podría tomar sin que estén especificados.

- Ejemplo para una consulta:

Flujo Principal	Flujo Alternativo
...	
3. El sistema realiza la búsqueda de todos aquellos clientes registrados en el sistema. - Si hay resultados de búsqueda, se muestran de la siguiente manera: - Si no hay resultados de búsqueda, se muestra el siguiente mensaje: ...	
...	

Como vemos en este ejemplo, el hecho de que no haya resultados de búsqueda no constituye un error o un fallo (flujo alternativo), sino que es algo totalmente esperado y normal dentro de la funcionalidad.

- Ejemplo para la modificación de un producto:

Flujo Principal	Flujo Alternativo
1. El caso de uso comienza cuando el usuario presiona “Modificar Producto” sobre un producto seleccionado.	
2. Si el producto seleccionado no tiene stock, el sistema muestra el siguiente mensaje: “El producto que se está modificando no posee stock”. El mensaje permanece visible por 5 segundos, y luego se cierra automáticamente.	
3. El sistema muestra los campos del producto a modificar: ...	
4. ...	

Aquí nuevamente se puede ver la evaluación de una condición que no ocasiona un error en el sistema, sino que es algo esperado en la ejecución.

Ciclos

Muchas veces se necesitará especificar ciclos. Esto es, recorrer una colección de datos, y realizar alguna tarea. Para representarlos se puede optar por la utilización de cualquiera de las estructuras conocidas en los lenguajes de programación:

- while
- do...while
- for / foreach
- etc.

Estos constructores serán representados en lenguaje coloquial (es decir, lenguaje natural, no pseudocódigo). Lo importante es especificar de forma no ambigua los elementos sobre los cuales se va a iterar. A continuación se muestran algunos ejemplos de ciclos utilizando el equivalente a un *for / foreach*:

- Ejemplo para el envío de mails de marketing:

Flujo Principal	Flujo Alternativo
1. ...	
2. El sistema busca todos los clientes activos registrados.	
3. Para cada cliente obtenido en el paso anterior, el sistema: a. obtiene el correo electrónico del cliente. b. obtiene la oferta que mejor se ajusta a las preferencias del cliente. c. envía un correo electrónico al cliente con la oferta seleccionada.	
4. ...	

Aquí la expresión “Para cada cliente obtenido en el paso anterior” es equivalente a:

foreach (cliente in listaClientes)

representando así el ciclo de forma no ambigua.

Campos a completar por el usuario

En muchos casos de uso, sobre todo aquellos que describen altas o modificaciones de entidades, se necesitará describir un formulario (campos) que el usuario debe completar.

En relación con el tema de atributos de calidad de requerimientos, y teniendo en cuenta que las especificaciones de los casos de uso son utilizadas por los programadores para implementar el sistema, es importante eliminar las ambigüedades y especificar de forma completa el caso de uso. Por ejemplo, el campo "nombre del producto", que seguramente en algún momento se utilizará en algún caso de uso. Se puede documentar así:

Flujo Principal	Flujo Alternativo
1. El caso de uso comienza cuando el usuario presiona "Agregar Producto".	
2. El sistema muestra una pantalla con los siguientes campos a completar: • nombre del producto • ...	
3. ...	

Claramente, este caso de uso es ambiguo, dado que puede tener más de una interpretación:

- El programador puede pensar que el campo no es obligatorio, dado que la especificación no dice nada al respecto.
- El tester puede pensar que el campo es obligatorio, dado que sin un nombre, no se puede describir a un producto.

En pos de cumplir con los atributos de calidad de la especificación de requerimientos, se debe detallar el formato de los campos a completar:

- si son obligatorios o no.
- si son cadenas de caracteres, definir los caracteres aceptados, y su máxima longitud.
- si son campos de tipo calendario, definir si se aceptan fechas del pasado, del futuro, a partir de qué fecha, etc.
- si son listas desplegables (más detalle en la próxima sección), definir qué elementos la componen, y definir además qué campo se muestra de cada elemento. Por ejemplo, si es una lista desplegable de proveedores, definir qué campo del proveedor se muestra en la lista.

Algunos ejemplos:

- Nombre: Campo de texto, obligatorio, máximo 255 caracteres.
- Marca: lista desplegable, obligatoria, contiene todas las marcas registradas en el sistema. De cada marca se muestra el nombre.
- Precio Inicial: campo de tipo moneda, obligatorio.
- Stock Inicial: campo de tipo numérico, obligatorio, solo permite enteros positivos.
- Observaciones: campo opcional, tipo texto. Máximo de 2048 caracteres.
- Fecha de nacimiento: campo obligatorio de tipo calendario, sólo permite seleccionar fechas hasta el día de hoy.

Algo que vale la pena mencionar aquí es la importancia de mantener la consistencia en los campos. Es decir, tomando el ejemplo de la *Marca*, si se pide que de cada marca se muestre el nombre, es necesario que la marca tenga ese atributo. Entonces, si existe la funcionalidad de *Agregar Marca*, se debe poder completar ahí el campo *nombre*.

Campos que el usuario puede seleccionar

A medida que se van identificando funcionalidades, hay que tener en cuenta que el sistema debe brindar asistencia al ejecutarlas. Muchas veces se ingresarán datos al sistema, pero de una manera distinta.

Por ejemplo, en la generación de un pedido a un proveedor. Seguramente, en este caso de uso se tendrá que indicar al sistema a qué proveedor en particular le queremos hacer un pedido. Un error muy común en estos casos es que, al momento de especificar esto, se tome al proveedor como un campo de texto. Esto es incorrecto, dado que el dato ya se encuentra en el sistema.

Otro ejemplo típico es la selección de una provincia. El error más común aquí es que el usuario cuente con un campo de texto para escribir la provincia en cuestión. Pero esto es incorrecto, dado que puede ingresarse de diversas maneras:

- Santa Fe
- santa fe
- sta. fe

Es necesario tener en cuenta que, si el usuario debe ingresar un dato que *ya existe* en el sistema, debe hacerlo seleccionando el dato. Para esto, por lo general, vamos a usar **algún campo que permita la selección de una o más opciones**. Tomando como ejemplo la creación de un producto:

- Categoría: campo de selección de una opción que contiene todas las categorías registradas en el sistema. De cada categoría se muestra el nombre, ordenadas alfabéticamente.

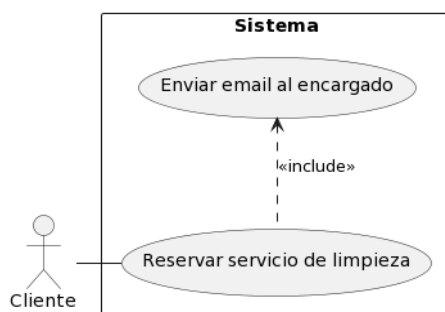
Inclusión de casos de uso

Normalmente, en cualquier sistema se encontrarán partes de funcionalidades que se repiten. Para evitar replicar estas funcionalidades comunes, podemos extraerlas como casos de uso, y luego incluir estas nuevas funcionalidades en aquellos casos de uso que sean necesarios. Esto es similar a lo que ocurre con la invocación de un método común en un lenguaje de programación.

Para describir esta situación se utilizará la siguiente sintaxis:

include(nombre del caso de uso incluido)

Por ejemplo:



En la plantilla del caso de uso *Reservar servicio de limpieza* se documentará lo siguiente dentro del flujo principal cuando se quiera invocar al caso de uso incluido:

Flujo Principal	Flujo Alternativo
1. ...	
2. ...	
3. include(Enviar email al encargado)	
4. ...	

No hay ninguna restricción en cuanto a dónde colocar el include, puede ser en cualquier paso del flujo principal o alternativo.

Incluye y los actores

Muchas veces los casos de uso comunes que son incluidos no se relacionan con ningún actor en el diagrama. Entonces, ¿quiénes serían los actores primarios en ese caso?

Tomemos como ejemplo lo mostrado en la figura anterior. *Enviar email al encargado* es activado solamente por el caso de uso *Reservar servicio de limpieza*. El actor *Cliente* no debe considerarse actor principal del caso de uso incluido, dado que no lo activa directamente. El caso de uso incluido quedaría así:

ID	002
Nombre	Enviar email al encargado
Breve descripción	...
Prioridad	...
Complejidad	...
Actor principal	
Actores secundarios	

Si el caso de uso *Enviar email al encargado* fuese activado directamente por otro actor, ese sí aparecería como actor principal.

Puntos de extensión

Los puntos de extensión son aquellos lugares en los que el caso de uso permite ser ampliado en su comportamiento por otros casos de uso. Se puede pensar en los *puntos de extensión* como aquellos *hooks* (“ganchos”) que provee un caso de uso, desde donde pueden “engancharse otras funcionalidades”.

Típicamente, estos puntos de extensión se utilizarán como aquellas opciones que tiene el actor para poder realizar funcionalidades fuera del alcance original del caso de uso en cuestión, comportamiento adicional opcional que puede ser incorporado en ese punto. Un ejemplo típico de esto es la consulta de alguna entidad, por ejemplo, la consulta de categorías:

Consultar Categorías

Nombre:
Descripción:
Torneo:

Buscar

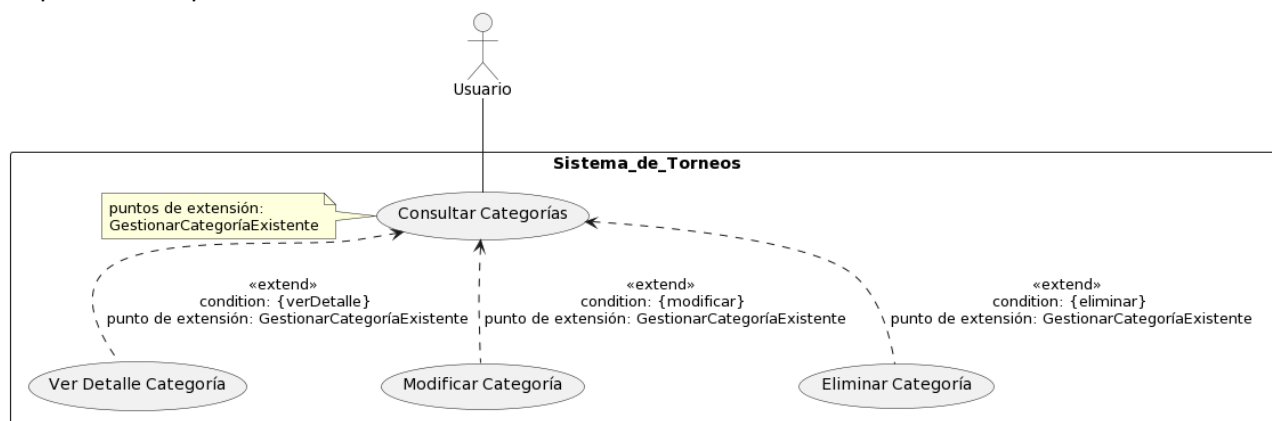
Limpiar

Resultados de la Búsqueda

Nombre	Torneo	Categoría superior	Acciones
A	Clausura 2012		Ver Detalle Modificar Eliminar
B	Clausura 2012	A	Ver Detalle Modificar Eliminar
C	Clausura 2012	B	Ver Detalle Modificar Eliminar

< 1 2 3 > Página 1 de 3

Cómo se ve en la imagen, al *Consultar Categorías*, el usuario puede acceder a *Ver Detalle*, *Modificar* o bien *Eliminar* cada *Categoría*. Estas funcionalidades extienden el caso de uso *Consultar Categorías*, dado que amplían su comportamiento:



Para representar esto en la especificación hay que tener en cuenta que los casos de uso que forman parte de la relación de extensión deben ser independientes. Para el ejemplo introducido, desde la *Consulta de categorías* no podemos mencionar a ninguno de los otros 3 casos de uso que extienden. Sólo se debe indicar el punto de extensión. De la misma forma, desde los casos de uso que extienden, no podemos nombrar la *Consulta de Categorías*.

La especificación sería:

- Consultar Categorías:

Flujo Principal	Flujo Alternativo
1. ...	
2. ...	
3. El sistema muestra los resultados de búsqueda en un listado paginado. Para cada categoría se muestra: - Nombre - Torneo - Categoría superior Además, para cada categoría se muestran 3 botones: - Ver Detalle - Modificar - Eliminar El listado contiene 5 categorías por página, y debajo del listado se encuentran los controles para pasar de una página a otra. Punto de Extensión: "GestionarCategoríaExistente".	
4. ...	

Notar que el punto de extensión no nombra ninguno de los casos de uso que extienden a este caso de uso, simplemente identifica el punto donde el flujo puede ser extendido, mediante un punto de extensión.

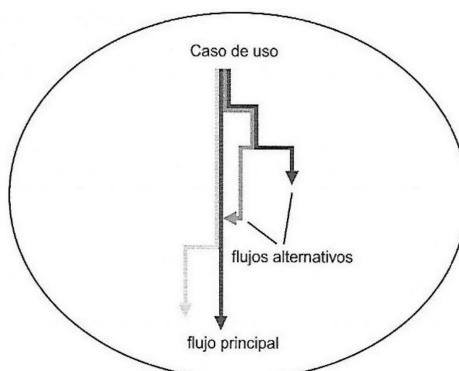
- Ver Detalle Categoría

Flujo Principal	Flujo Alternativo
1. El caso de uso comienza cuando el usuario presiona "Ver Detalle" sobre una categoría seleccionada.	
2. ...	
3. ...	

Lo mismo es aplicable a los otros casos de uso que extienden la consulta.

Flujo Alternativo

El flujo alternativo de un caso de uso es aquel comportamiento que provoca la desviación del flujo principal. La siguiente figura ilustra al flujo principal de un caso de uso y sus múltiples flujos alternativos:



Como se observa en la figura, esta desviación del flujo principal puede ser temporal o definitiva:

- Una desviación temporal sale del flujo principal, especifica el comportamiento alternativo y luego vuelve al flujo principal (ya sea antes o después del momento del desvío). Por ejemplo, un paso de un caso de uso donde se valida el formato de datos ingresados y falla la validación. En este ejemplo, el usuario puede volver al flujo principal y editar los datos que ingresó.
- Una desviación permanente denota una salida del flujo principal, no volviendo a éste. Cabe destacar que el estado resultante del sistema debe estar definido como una postcondición del caso de uso, de éxito o fracaso. Un ejemplo típico es una validación que falla, pero que no depende del ingreso de datos del usuario. Por ejemplo, si durante la ejecución de un caso de uso, es necesario validar información utilizando otro sistema (como podría ser una entidad bancaria) pero este se encuentra caído o no responde, entonces la ejecución no puede continuar, se produce una falla que no es recuperable.

A continuación, se describe un ejemplo para cada caso:

- Desviación temporal:

ID	1
Nombre	Reservar servicio de limpieza
Breve descripción	Permite al cliente reservar un trabajo de limpieza en el lugar que elija.
Flujo Principal	Flujo Alternativo
1- El cliente presiona el botón "Reservar Servicio"	
2- El sistema muestra los siguientes campos a completar: - Fecha: campo tipo calendario, obligatorio. - Horario: campo de tipo horario, obligatorio. - Domicilio: campo de texto, máximo de 255 caracteres, obligatorio. - Datos significativos: área de texto, máximo 4096 caracteres, opcional. Debajo se muestra el botón "Reservar".	
3- El cliente completa los campos y presiona "Reservar".	
4- El sistema valida que los campos cumplan con el formato requerido.	4- La validación falla: 4.1 - El sistema vuelve al paso 2 para que el usuario reingrese los datos. Debajo de cada campo erróneo, se muestra el formato requerido.
5- ...	

Aquí se ve cómo desde el flujo alternativo 4.1 se vuelve al flujo principal, al paso 2, para que el usuario pueda reingresar los datos.

- Desviación permanente:

ID	CU 001	
Nombre	Armar Pedido	
Breve descripción	...	
Prioridad	...	
Complejidad	...	
Actor principal	...	
Actores secundarios	...	
Precondiciones	...	
Postcondiciones	Éxito: El pedido queda marcado como “En armado”, asignado al usuario.	
	Fracaso: No se realiza ninguna modificación sobre el pedido.	
Flujo Principal		Flujo Alternativo
1. ...		
2. ...		
3- El sistema busca el pedido seleccionado para armar.		
4- El sistema valida que: - El pedido esté en estado “Sin Armar”		4- El pedido no está en estado “Sin Armar” 4.1- El sistema muestra un mensaje “El pedido está siendo armado por otro empleado”. 4.2- El usuario presiona Aceptar. 4.3- Finaliza el CU.
5. ...		
Flujos de excepción		
Observaciones		

En este ejemplo se ve que, al estar el pedido siendo armado por otro empleado, no se puede continuar con el caso de uso, el error no es recuperable.

Cabe destacar que, para los dos ejemplos previos, estos errores son en realidad chequeos que el sistema hace. Hay que tener en cuenta que, si un chequeo falla, no significa que el caso de uso falla. El mismo está preparado para manejar tanto las condiciones del flujo principal, como aquellas del flujo alternativo. Es por esto que lo que se encuentre dentro de los dos flujos, debe ser considerado como “normal”, a los efectos del funcionamiento del sistema.

Flujos de excepción

En esta plantilla los flujos de excepción se consideran como aquellos flujos que, similarmente a lo que sucede con los alternativos, salen del flujo principal de forma permanente, pero, además, pueden ocurrir en distintos

pasos del mismo. Un ejemplo frecuente de esto es la posibilidad que tiene el usuario de cancelar la ejecución de una funcionalidad:

Flujos de excepción	El usuario puede cancelar la operación cuando lo desee, presionando el botón "Cancelar". Luego de esto, el sistema muestra el mensaje "La operación de agregar un producto ha sido cancelada". El usuario luego presiona "Aceptar" y finaliza el caso de uso.
----------------------------	---

Cabe destacar que el flujo de excepción ocasiona que sea cierta alguna de las postcondiciones de fracaso.

Observaciones

Con la plantilla para CUs analizada, se logra cubrir en gran medida las necesidades de información. Sin embargo, puede existir información que no esté considerada en los campos descritos de la misma. Para esta situación puede utilizarse el campo Observaciones. En este campo puede documentarse información específica del CU, funcionalidad, solicitante, etc., que aumente la comprensión del mismo. Por ejemplo, se puede especificar que un departamento o área en particular pidió esa funcionalidad, entonces es posible documentar: "Esta funcionalidad fue pedida por el departamento de Ventas".

Bibliografía

- Arlow, Jim; Neustadt, Ila. "UML 2", 2006.
- Karl Wiegers; Joy Beatty. Software Requirements, Third Edition, 2013.